GenAI Documentation

Everyone must complete this section. If not GenAI was used, write, “No GenAI used for this task.” When GenAI is not used, process evidence should still demonstrate iteration, revision, or development over time.

Because GenAI can closely mimic human-created work, instructors or TAs may occasionally request additional process evidence to confirm non-use. This may include original working files (e.g., an illustrator file), intermediate drafts, or a brief check-in with a TA to walk through your process.

These requests are not an assumption of misconduct. They are part of ensuring academic integrity in an environment where distinguishing between human-created and AI-generated work is increasingly difficult.

If GenAI was used (keep each response as brief as possible):

Date Used: March 8, 2026

Tool Disclosure: Claude

Purpose of Use: I had GenAI look at my current p5.js sketch structure and create/edit the Week 6 side quest of adding synthesized sound feedback (p5.Oscillator) + reactive physics (particle bursts, screen shake) upon gem collection, and gameplay polish (respawning gems, restart system, HUD).

Summary of Interaction: Claude reviewed my game files and diagnosed that audio, gravity, and collisions weren't turned on yet. It recommended a pickup gem system with three concurrent feedback channels (audio clip, particle explosion, camera shake). It offered the entire implementation up front, then I asked Claude for more spitshine: gems respawning with delay from random positions, press R to retry last level, display wave count, keep highscore. Claude sent back an updated implementation with everything.

Human Decision Point(s): I limited the scope to just collecting gems rather than other features Claude could have implemented (enemies, hazards, platforms). I opted to include the respawn and restart systems after playtesting version 1, as it felt too short to only have four stationary gems. I kept it using synthesized oscillators rather than importing external audio files, as it kept everything contained for easier submission.

Integrity & Verification Note: I validated each version by previewing the sketch locally in my browser, ensuring that the fox sprite loaded and animated correctly, that gravity and collision with the ground still worked, that gems activated all three feedback responses when overlapping, and that the respawn timer and restart key still worked correctly. I checked that the p5.sound library was loading by inspecting the browser console for errors.

Scope of GenAI Use: GenAI coded up the gem system, particle engine, pickup sound fx function, screen shake, respawn timer, restart logic, and HUD. The sprite sheet, animation blueprints, and initial project structure were all provided by the instructors. All choices for feature/build, iterate, or polish request were mine.

Limitations or Misfires: Claude's first parse thought I had multiple files/classes for my project because I mentioned it in my first prompt line (Game, Level, SoundManager). However, when it viewed my actual files it realized that I had written everything in one single file sketch. Also my first version just spawned gems at set positions which got boring so I edited my prompt to mention random spawns.

Summary of Process (Human + Tool)

Claude and I began with my sprite animation example given by the instructor (no physics, no sound, no gems). I told Claude my Week 6 goals and had him analyze my files before starting to code. Claude's first pass included gravity, ground platform, realistic player movement, gem sprites, and all three feedback pieces (sound, particles, shake). After gameplay testing, I wanted more longevity so I asked Claude for a second pass that included gem respawn, random spawn positions, key to restart, wave count, and high score. I locally tested Claude's final file to ensure everything fit.

Decision Points & Trade-offs

I chose to use a synthesized oscillator for the pickup sound instead of loading an audio file because it avoided asset management and allowed my project to remain portable enough to submit. I chose to keep gems as static colliders with a visual representation of floating, instead of dynamic physics bodies because the dynamic gems fell off screen and added unnecessary complexity.

Verification & Judgement

I tested each change by loading the sketch in my browser. I confirmed that the fox sprite animated and moved with gravity as before, that gems produced all three types of feedback upon collision (audible blip, visible particles, screen shake), that new waves generated upon collection with the flash indicator, and that hitting R reset the state without lingering objects or effects. I compared my finished work to the Week 6 assignment to ensure it clearly showed both sound and physics/reactive motion at play.

Appendix

User:

I am working on a Week 6 p5.js side quest for a game design class.

Assignment goal: Add sound or simple physics to my sketch, for example, visual effects triggered by motion, collisions, or player actions. Bonus: combine both sound and physics for a reactive, multi-sensory effect.

Important context: This is not a one-file beginner P5 sketch. My project already has a modular architecture. My entry file is main.js, and it is only supposed to handle the VIEW/orchestration layer, setup/draw lifecycle, asset boot, camera, HUD compositing, and system wiring. Gameplay rules and entity behaviour should NOT be added directly into main.js unless absolutely necessary. The code comments in main.js explicitly say: main.js owns VIEW setup, Game owns WORLD orchestration, gameplay logic lives in Level/entities, main.js should not implement entity behaviour or gameplay rules.

The project already includes p5.js, p5.sound, and p5play/planck physics, so sound and simple physics are both available.

Please help me implement a Week 6 side quest feature that combines BOTH: (1) sound feedback, and (2) a simple physics or reactive motion effect. I want the feature to feel polished but still realistic for a student side quest, not like a huge game redesign.

Before writing code: First, inspect the project structure and identify the correct files to edit. Look through src/, data/, and any gameplay-related files before suggesting changes. Respect the current architecture and separation of responsibilities. Do NOT dump all logic into main.js if it belongs in Game, Level, Player, entity, collision, or SoundManager code.

Feature requirements: Please design and implement one small feature that clearly satisfies the assignment. Examples include: when the player collides with a special object/collectible/hazard, play a sound effect and trigger a reactive visual/physics response; or when the player jumps/lands/dashes/attacks, play sound and cause a bounce, recoil, knockback, screen shake, particle burst, or other simple physical reaction; or when an object is hit, it gets pushed, bounces, or emits particles while playing a sound.

I want the final result to feel responsive, give clear player feedback, be easy to explain in an assignment submission, fit my existing visual style and code architecture, and avoid unnecessary complexity.

Constraints: Keep the existing game working. Do not rewrite the whole project. Reuse existing systems where possible, especially SoundManager and existing collision/update logic. Keep

code readable and well-commented. Follow the style already used in the project. If adding assets is necessary, explain exactly where to place them and how to load them. If there are already assets/sounds that can be reused, prefer those. If no new sound files exist, provide a fallback approach and explain it clearly.

Please proceed in the following order: (1) Project Analysis, (2) Feature Choice, (3) Implementation Plan, (4) Code Changes, (5) Explanation, (6) Debugging Notes.

Claude:

I will begin by inspecting the project structure to understand the codebase before making any recommendations.

After reviewing the project files (sketch.js, index.html, style.css, jsconfig.json), here is the full analysis and implementation.

STEP 1 — PROJECT ANALYSIS

The project is a single-file p5.js sketch (sketch.js) using p5play with Planck physics loaded via CDN. The current state is as follows:

Gameplay and rendering both live in sketch.js. The player sprite has four animations (idle, run, jump, attack) but no actual movement logic — the controls only swap animations without changing velocity or position.

Sound is not currently loaded. The index.html includes p5, planck, and p5play, but not the p5.sound library. No sounds are used anywhere.

Physics: p5play/Planck is available but gravity is set to 0, there is no ground, and player.removeColliders() is called, so physics collisions are effectively disabled.

Collision and physics reactions do not exist yet. This is exactly what Week 6 asks us to add.

Since this is a single-file sketch, all changes will go in sketch.js and index.html. That is architecturally correct for this project.

STEP 2 — FEATURE CHOICE

Feature: "Collectible Gems with Pickup Sound + Particle Burst + Screen Shake"

When the player walks into a gem, three things happen simultaneously: (1) a synthesized pickup sound plays using p5.Oscillator (no audio files needed), (2) a burst of colored particles spawns at the gem's position (reactive motion), and (3) a brief screen shake gives a sense of physical impact.

This is a strong choice because it clearly combines sound and physics/motion (satisfying Week 6), adds real player movement and gravity (making the sketch actually playable), uses p5play's built-in collision system, requires no external audio files, is visually impressive while only adding roughly 80 lines of code, and is easy to explain in a reflection.

STEP 3 — IMPLEMENTATION PLAN

index.html (required): Add the p5.sound CDN script tag, since p5.sound is not currently loaded and is needed for synthesized sound.

sketch.js (required): Enable gravity and add a ground platform so the player can walk and jump. Add actual movement logic using velocity for existing key controls. Create a group of gem sprites the player can collect. Add a playPickupSound() function using p5.Oscillator. Implement a simple particle system using an array of objects updated each frame. Add screen shake logic via a small random camera offset that decays. Implement overlap detection so that when the player contacts a gem, it triggers the sound, particles, and shake simultaneously. Add a score HUD.

STEP 4 — CODE CHANGES

index.html — add the p5.sound CDN (one line added):

```
html<!doctype html>

<html lang="en">

  <head>

    <meta charset="utf-8" />

    <title>GBDA302 Week 6 - Side Quest: Sound + Physics</title>

    <style>

      body {

        margin: 0;

        overflow: hidden;

        background: #222;
```

```
      }
      canvas {
        display: block;
      }
    </style>
  </head>
  <body>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/p5.min.js"></script>
    <script src="https://cdn.jsdelivr.net/npm/p5@1.11.4/lib/addons/p5.sound.min.js"></script>
    <script src="https://p5play.org/v3/planck.min.js"></script>
    <script src="https://p5play.org/v3/p5play.min.js"></script>
    <script src="sketch.js"></script>
  </body>
</html>
```

sketch.js — full replacement:

```
js/*
  Week 6 — Side Quest: Sound + Physics Feedback
  Base: Example 1 (Sprites, Sprite Sheets, & Animation)

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026

  What's new (Week 6 additions):
    - Gravity + ground platform make the world physical
```

```
  - Player can now move and jump with real velocity
  - Collectible gems trigger THREE reactive feedback effects:
      1. Synthesized pickup sound (p5.Oscillator)
      2. Particle burst at the gem's position
      3. Brief screen shake for impact feeling
  - Score HUD tracks gems collected


 Controls:
  A / D  or  Left / Right Arrow    Horizontal movement
  W  or  Up Arrow                  Jump
  S  or  Down Arrow                Idle
  Space Bar                        Attack
*/


// ─── SPRITES & IMAGES ──────────────────────────────────────────────────────
let player;
let playerImg;
let ground;
let gems; // p5play Group for collectible gems


let playerAnis = {
  idle:   { row: 0, frames: 4, frameDelay: 10 },
  run:    { row: 1, frames: 4, frameDelay: 3 },
  jump:   { row: 2, frames: 3, frameDelay: 8, frame: 0 },
```

```
  attack: { row: 3, frames: 6, frameDelay: 2 },
};

// ─── LEVEL CONSTANTS ──────────────────────────────────────────────────
const VIEWW = 320, VIEWH = 180;
const FRAME_W = 32, FRAME_H = 32;
const GRAVITY = 10;
const GROUND_Y = VIEWH - 12;
const PLAYER_SPEED = 1.5;
const JUMP_FORCE = -3.8;

// ─── WEEK 6: FEEDBACK SYSTEMS ─────────────────────────────────────────
let particles = [];
let shakeAmount = 0;
let score = 0;
let pickupOsc;
let oscReady = false;

// ─── PRELOAD ──────────────────────────────────────────────────────────
function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
}

// ─── SETUP ────────────────────────────────────────────────────────────
```

```
function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  allSprites.pixelPerfect = true;
  world.gravity.y = GRAVITY;

  ground = new Sprite(VIEWW / 2, GROUND_Y, VIEWW, 16, "static");
  ground.color = color(60, 140, 60);
  ground.stroke = color(40, 100, 40);

  player = new Sprite(VIEWW / 2, GROUND_Y - 30, FRAME_W, FRAME_H);
  player.spriteSheet = playerImg;
  player.rotationLock = true;
  player.anis.w = FRAME_W;
  player.anis.h = FRAME_H;
  player.anis.offset.y = -4;
  player.addAnis(playerAnis);
  player.ani = "idle";
  player.w = 18;
  player.h = 20;
  player.friction = 0;
  player.bounciness = 0;

  gems = new Group();
  gems.diameter = 8;
```

```
  gems.color = color(255, 220, 50);
  gems.stroke = color(200, 160, 0);
  gems.strokeWeight = 1;
  gems.collider = "static";
  gems.bounciness = 0;

  spawnGems();
}

function spawnGems() {
  let gemPositions = [
    { x: 60,  y: GROUND_Y - 30 },
    { x: 130, y: GROUND_Y - 50 },
    { x: 200, y: GROUND_Y - 30 },
    { x: 260, y: GROUND_Y - 45 },
  ];
  for (let pos of gemPositions) {
    let g = new gems.Sprite(pos.x, pos.y);
    g._baseY = pos.y;
  }
}

function playPickupSound() {
  if (getAudioContext().state !== "running") {
```

```
    getAudioContext().resume();
  }
  if (!pickupOsc) {
    pickupOsc = new p5.Oscillator("sine");
    pickupOsc.amp(0);
    pickupOsc.start();
    oscReady = true;
  }
  pickupOsc.freq(440);
  pickupOsc.amp(0.3, 0.01);
  setTimeout(() => { if (pickupOsc) pickupOsc.freq(880, 0.08); }, 60);
  setTimeout(() => { if (pickupOsc) pickupOsc.amp(0, 0.1); }, 150);
}

function spawnParticles(x, y) {
  let count = 12;
  for (let i = 0; i < count; i++) {
    let angle = random(TWO_PI);
    let speed = random(0.5, 2.5);
    particles.push({
      x: x, y: y,
      vx: cos(angle) * speed,
      vy: sin(angle) * speed - 1,
      life: 1.0,
```

```
      size: random(2, 4),
      col: color(random(220, 255), random(180, 230), random(30, 80)),
    });
  }
}

function updateParticles() {
  for (let i = particles.length - 1; i >= 0; i--) {
    let p = particles[i];
    p.x += p.vx;
    p.y += p.vy;
    p.vy += 0.05;
    p.life -= 0.03;
    if (p.life <= 0) { particles.splice(i, 1); continue; }
    push();
    noStroke();
    let c = color(red(p.col), green(p.col), blue(p.col), p.life * 255);
    fill(c);
    ellipse(p.x, p.y, p.size * p.life, p.size * p.life);
    pop();
  }
}

function triggerShake(intensity) {
```

```
  shakeAmount = intensity;
}

function draw() {
  if (shakeAmount > 0.1) {
    translate(random(-shakeAmount, shakeAmount), random(-shakeAmount, shakeAmount));
    shakeAmount *= 0.85;
  } else {
    shakeAmount = 0;
  }

  background("skyblue");

  for (let g of gems) {
    g.y = g._baseY + sin(frameCount * 0.05 + g.x) * 2;
  }

  player.overlaps(gems, collectGem);

  let isOnGround = player.colliding(ground);

  if (kb.pressing("left") || kb.pressing("a")) {
    player.vel.x = -PLAYER_SPEED;
    if (isOnGround) player.ani = "run";
```

```
  player.mirror.x = true;
} else if (kb.pressing("right") || kb.pressing("d")) {
  player.vel.x = PLAYER_SPEED;
  if (isOnGround) player.ani = "run";
  player.mirror.x = false;
} else {
  player.vel.x = 0;
  if (isOnGround && player.ani.name !== "attack") {
    player.ani = "idle";
  }
}

if ((kb.presses("up") || kb.presses("w")) && isOnGround) {
  player.vel.y = JUMP_FORCE;
  player.ani = "jump";
}

if (kb.presses(" ")) {
  player.ani = "attack";
}

player.x = constrain(player.x, 10, VIEWW - 10);

updateParticles();
```

```
  push();
  resetMatrix();
  fill(255);
  stroke(0);
  strokeWeight(0.5);
  textSize(10);
  textAlign(LEFT, TOP);
  text("Gems: " + score, 6, 6);
  pop();
}

function collectGem(player, gem) {
  playPickupSound();
  spawnParticles(gem.x, gem.y);
  triggerShake(3);
  score++;
  gem.remove();
}
```

STEP 5 — EXPLANATION

The sound is triggered when the player's sprite overlaps a gem via p5play's overlap callback. The playPickupSound() function creates a sine wave oscillator that sweeps from 440Hz to 880Hz and fades out in roughly 150ms, producing a cheerful rising blip with no audio files required.

The particle burst spawns 12 small objects with random radial velocities and a slight upward bias. Each frame, particles move, fall with mini-gravity, and fade out. Screen shake applies a small random camera offset that decays by 85% per frame, fizzling out in about 10 frames.

The combined effect makes gem collection feel punchy and satisfying — the player hears the blip, sees the gold sparkles fly outward, and feels the screen jolt simultaneously.

This satisfies Week 6 by combining synthesized sound (p5.Oscillator), particle physics (velocity, gravity, decay), and screen shake (camera displacement), all triggered reactively by a player collision event.

STEP 6 — DEBUGGING NOTES

Audio may not play on the very first gem collection if the user has not yet clicked the page, due to browser autoplay restrictions. The code calls getAudioContext().resume() to handle this, but the second gem onward will always work.

Ensure assets/foxSpriteSheet.png is still in the correct relative path. The original removeColliders() call has been removed — if it is accidentally left in, overlaps will not fire. Particles will appear pixelated, which is expected and matches the canvas rendering mode.

If p5.Oscillator throws a constructor error, the p5.sound CDN was not loaded. Verify the script tag order in index.html.

STEP 7 — SUBMISSION REFLECTION

For my Week 6 side quest, I added a gem collection system that combines sound feedback with reactive physics effects. When the player's fox sprite overlaps a collectible gem, three things happen simultaneously: a synthesized rising tone plays using p5's Oscillator, a burst of gold particles flies outward with simulated mini-gravity, and the screen shakes briefly to give a sense of physical impact. These layered feedback signals make the interaction feel responsive and satisfying, giving the player clear confirmation that something meaningful happened. This supports inclusive design by providing both audio and visual feedback channels, ensuring accessibility across different player needs.

User:

Can you make this game a bit more fleshed out without doing anything drastic? For example, maybe a restart button, or gems that keep spawning over time.

Claude:

Here is the updated sketch.js with the following additions:

Gem respawn system: every 4 seconds, a new wave of gems spawns at random positions (only if fewer than 4 gems exist on screen).

Press R to restart: resets score, clears particles, respawns gems, and repositions the player.

Wave counter: tracks how many waves of gems have been collected.

High score: persists across restarts within the same session.

"New Gems!" flash: brief text appears when a new wave spawns to alert the player.

index.html remains unchanged from the previous version.

sketch.js — full replacement:

```
js/*
  Week 6 — Side Quest: Sound + Physics Feedback
  Base: Example 1 (Sprites, Sprite Sheets, & Animation)

  Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han
  Date: Feb. 26, 2026

  What's new (Week 6 additions):
    - Gravity + ground platform make the world physical
    - Player can now move and jump with real velocity
    - Collectible gems trigger THREE reactive feedback effects:
        1. Synthesized pickup sound (p5.Oscillator)
        2. Particle burst at the gem's position
        3. Brief screen shake for impact feeling
    - Gems respawn every few seconds in new positions
    - Press R to restart at any time
```

```
    - Score + high score + wave counter HUD


  Controls:

    A / D  or  Left / Right Arrow    Horizontal movement

    W  or  Up Arrow                  Jump

    Space Bar                        Attack

    R                                Restart

*/


// ─── SPRITES & IMAGES ──────────────────────────────────────────────

let player;

let playerImg;

let ground;

let gems;


let playerAnis = {

  idle:   { row: 0, frames: 4, frameDelay: 10 },

  run:    { row: 1, frames: 4, frameDelay: 3 },

  jump:   { row: 2, frames: 3, frameDelay: 8, frame: 0 },

  attack: { row: 3, frames: 6, frameDelay: 2 },

};


// ─── LEVEL CONSTANTS ───────────────────────────────────────────────

const VIEWW = 320, VIEWH = 180;
```

```
const FRAME_W = 32, FRAME_H = 32;
const GRAVITY = 10;
const GROUND_Y = VIEWH - 12;
const PLAYER_SPEED = 1.5;
const JUMP_FORCE = -3.8;

// ─── WEEK 6: FEEDBACK SYSTEMS ──────────────────────────────────────
let particles = [];
let shakeAmount = 0;

// ─── GAME STATE ────────────────────────────────────────────────────
let score = 0;
let highScore = 0;
let wave = 1;
let lastSpawnTime = 0;
const RESPAWN_INTERVAL = 4000;
let spawnFlashTimer = 0;

let pickupOsc;

// ─── PRELOAD ───────────────────────────────────────────────────────
function preload() {
  playerImg = loadImage("assets/foxSpriteSheet.png");
}
```

```
// ─── SETUP ──────────────────────────────────────────────────────────────
function setup() {
  new Canvas(VIEWW, VIEWH, "pixelated");
  allSprites.pixelPerfect = true;
  world.gravity.y = GRAVITY;

  ground = new Sprite(VIEWW / 2, GROUND_Y, VIEWW, 16, "static");
  ground.color = color(60, 140, 60);
  ground.stroke = color(40, 100, 40);

  createPlayer();

  gems = new Group();
  gems.diameter = 8;
  gems.color = color(255, 220, 50);
  gems.stroke = color(200, 160, 0);
  gems.strokeWeight = 1;
  gems.collider = "static";
  gems.bounciness = 0;

  spawnGems();
  lastSpawnTime = millis();
}
```

```
function createPlayer() {
  player = new Sprite(VIEWW / 2, GROUND_Y - 30, FRAME_W, FRAME_H);
  player.spriteSheet = playerImg;
  player.rotationLock = true;
  player.anis.w = FRAME_W;
  player.anis.h = FRAME_H;
  player.anis.offset.y = -4;
  player.addAnis(playerAnis);
  player.ani = "idle";
  player.w = 18;
  player.h = 20;
  player.friction = 0;
  player.bounciness = 0;
}

function spawnGems() {
  let count = 4;
  let margin = 30;
  for (let i = 0; i < count; i++) {
    let gx = random(margin, VIEWW - margin);
    let gy = GROUND_Y - random(25, 55);
    let g = new gems.Sprite(gx, gy);
    g._baseY = gy;
```

```
  }
  spawnFlashTimer = 60;
}

function restartGame() {
  if (score > highScore) highScore = score;
  gems.removeAll();
  player.remove();
  createPlayer();
  score = 0;
  wave = 1;
  particles = [];
  shakeAmount = 0;
  spawnGems();
  lastSpawnTime = millis();
}

function playPickupSound() {
  if (getAudioContext().state !== "running") {
    getAudioContext().resume();
  }
  if (!pickupOsc) {
    pickupOsc = new p5.Oscillator("sine");
    pickupOsc.amp(0);
```

```
    pickupOsc.start();
  }
  pickupOsc.freq(440);
  pickupOsc.amp(0.3, 0.01);
  setTimeout(() => { if (pickupOsc) pickupOsc.freq(880, 0.08); }, 60);
  setTimeout(() => { if (pickupOsc) pickupOsc.amp(0, 0.1); }, 150);
}

function spawnParticles(x, y) {
  for (let i = 0; i < 12; i++) {
    let angle = random(TWO_PI);
    let speed = random(0.5, 2.5);
    particles.push({
      x: x, y: y,
      vx: cos(angle) * speed,
      vy: sin(angle) * speed - 1,
      life: 1.0,
      size: random(2, 4),
      col: color(random(220, 255), random(180, 230), random(30, 80)),
    });
  }
}

function updateParticles() {
```

```
  for (let i = particles.length - 1; i >= 0; i--) {
    let p = particles[i];
    p.x += p.vx;
    p.y += p.vy;
    p.vy += 0.05;
    p.life -= 0.03;
    if (p.life <= 0) { particles.splice(i, 1); continue; }
    push();
    noStroke();
    fill(red(p.col), green(p.col), blue(p.col), p.life * 255);
    ellipse(p.x, p.y, p.size * p.life, p.size * p.life);
    pop();
  }
}

function triggerShake(intensity) {
  shakeAmount = intensity;
}

function draw() {
  if (shakeAmount > 0.1) {
    translate(random(-shakeAmount, shakeAmount), random(-shakeAmount, shakeAmount));
    shakeAmount *= 0.85;
  } else {
```

```
  shakeAmount = 0;
}

background("skyblue");

for (let g of gems) {
  g.y = g._baseY + sin(frameCount * 0.05 + g.x) * 2;
}

if (gems.length === 0 && millis() - lastSpawnTime > RESPAWN_INTERVAL) {
  wave++;
  spawnGems();
  lastSpawnTime = millis();
}

player.overlaps(gems, collectGem);

let isOnGround = player.colliding(ground);

if (kb.pressing("left") || kb.pressing("a")) {
  player.vel.x = -PLAYER_SPEED;
  if (isOnGround) player.ani = "run";
  player.mirror.x = true;
} else if (kb.pressing("right") || kb.pressing("d")) {
```

```
  player.vel.x = PLAYER_SPEED;
  if (isOnGround) player.ani = "run";
  player.mirror.x = false;
} else {
  player.vel.x = 0;
  if (isOnGround && player.ani.name !== "attack") {
    player.ani = "idle";
  }
}

if ((kb.presses("up") || kb.presses("w")) && isOnGround) {
  player.vel.y = JUMP_FORCE;
  player.ani = "jump";
}

if (kb.presses(" ")) {
  player.ani = "attack";
}

if (kb.presses("r")) {
  restartGame();
}

player.x = constrain(player.x, 10, VIEWW - 10);
```

```
  updateParticles();

  drawHUD();
}

function collectGem(player, gem) {
  playPickupSound();
  spawnParticles(gem.x, gem.y);
  triggerShake(3);
  score++;
  gem.remove();

  if (gems.length === 0) {
    lastSpawnTime = millis();
  }
}

function drawHUD() {
  push();
  resetMatrix();

  fill(255);
  stroke(0);
```

```
strokeWeight(0.5);
textSize(10);
textAlign(LEFT, TOP);
text("Gems: " + score, 6, 6);
text("Wave: " + wave, 6, 18);

if (highScore > 0) {
  textAlign(RIGHT, TOP);
  text("Best: " + highScore, VIEWW - 6, 6);
}

textAlign(CENTER, BOTTOM);
fill(255, 255, 255, 150);
textSize(7);
text("R = Restart", VIEWW / 2, VIEWH - 4);

if (spawnFlashTimer > 0) {
  spawnFlashTimer--;
  let alpha = map(spawnFlashTimer, 60, 0, 255, 0);
  fill(255, 220, 50, alpha);
  stroke(0, 0, 0, alpha * 0.5);
  textSize(12);
  textAlign(CENTER, CENTER);
  text("New Gems!", VIEWW / 2, VIEWH / 2 - 20);
```

```
  }


  pop();

}
```

Changes from the previous version: gems now spawn at random positions each wave instead of fixed locations; a 4-second timer triggers new waves after all gems are collected; the wave counter increments each cycle; pressing R calls restartGame() which resets everything while preserving the high score; a "New Gems!" flash appears briefly when each wave spawns; and the HUD now displays the wave counter, high score, and a restart hint. full transcript of your conversation with the GenAI. GenAI transcripts are used for transparency and verification only and are not assessed for writing quality or completeness.